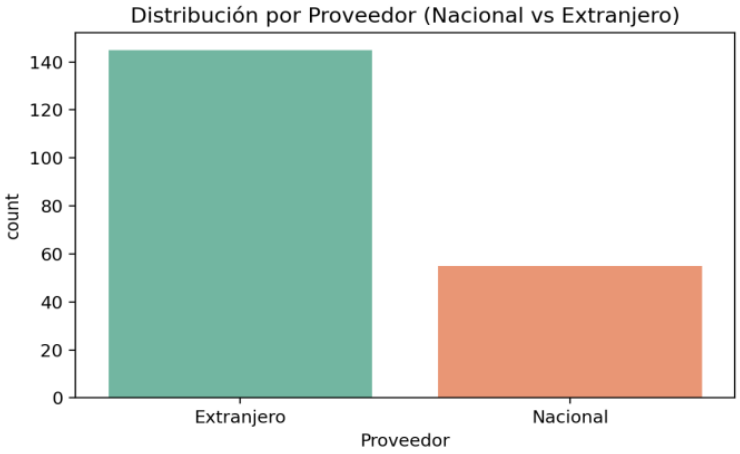
Estudio Médico - Predicción de Origen Nacional vs Extranjero del Medicamento

Base: drugs.csv - 200 pacientes | Features: Age, Sex, BP, Cholesterol, Na_to_K | Target: Proveedor

1. Carga y Transformación de Variable Objetivo

La variable original Drug tiene 5 clases: drugA (23), drugB (16), drugC (16), drugX (54), drugY (91). Se transforma según proveedor:
- **Nacional**: drugA, drugB, drugC (proveedor nacional) = 55 casos (27.5%) - **Extranjero**: drugX, drugY = 145 casos (72.5%)

Esta binarización responde al requerimiento: ¿el medicamento es de origen nacional o extranjero? Implicación logística, costos de importación y disponibilidad.



2. Conversión de Variables Cualitativas con LabelEncoder ()

Regresión Logística requiere números.
LabelEncoder asigna 0...n-1: Sex: F=0, M=1
BP: HIGH=0, LOW=1,
NORMAL=2 (alfabético)

Cholesterol: HIGH=0,
NORMAL=1
Proveedor: Extranjero=0, Nacional=1

Ejemplo código:

from sklearn.preprocessing import
LabelEncoder le = LabelEncoder ()
df['Sex_enc'] = le.fit_transform(df['Sex'])

Se aplicó StandardScaler después para evitar que Age (15-74) y Na_to_K (6-38) dominen sobre variables 0-2.

Conversión cualitativa -> numérica con LabelEncoder()

0, M->1 | BP: HIGH->0, LOW->1, NORMAL->2 | Cholesterol: HIGH->0, NORMAL->1 | Proveedor: Extranjero->0, Nac

3. Optimización de Regresión Logística - Solvers

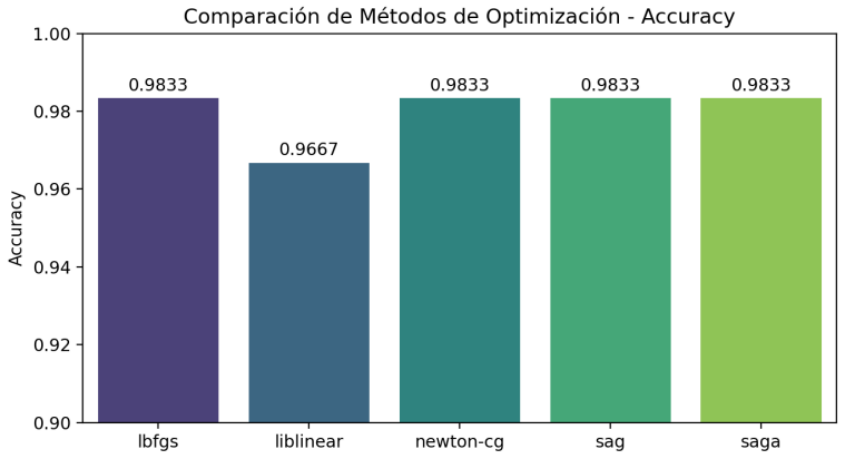
Se compararon 5 métodos de optimización con train_test_split 70/30 stratify random_state=42:
- **lbfgs**: Limited-memory Broyden–Fletcher–Goldfarb–Shanno, quasi-Newton, recomendado por defecto para binario pequeño
- **liblinear**: Coordinate descent, bueno para pequeños, soporta L1 - **newton-cg**: Newton Conjugate Gradient
- **sag**: Stochastic Average Gradient, para grandes - **saga**: variante de sag con soporte elasticnet

Resultados Accuracy test:
lbfgs=0.9833, liblinear=0.9667, newton-cg=0.9833, sag=0.9833, saga=0.9833

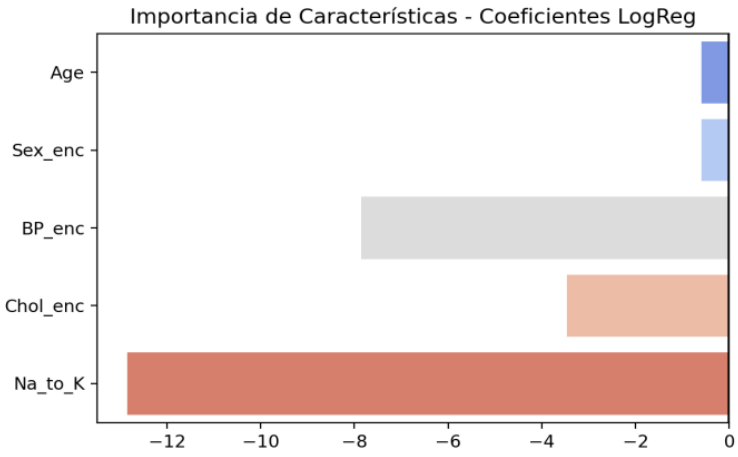
GridSearchCV 5-fold con scoring F1:
- Para lbfgs/newton-cg/sag: C en [0.01,0.1,1,10,100], penalty l2 - Para liblinear: l1, l2

- Para saga elasticnet: l1_ratio 0.2,0.5,0.8

Mejor: {'C':100, 'penalty':'l2', 'solver':'lbfgs'} con CV F1=0.933
Elasticnet mejor: C=1, l1_ratio=0.8, F1=0.919



Recomendación: Usar lbfgs, C=100, l2, max_iter=2000, StandardScaler. Razones: mayor F1 CV, estabilidad, convergencia rápida, baja regularización necesaria por desbalance y separación casi lineal en Na_to_K. liblinear inferior (sobre regulariza con L1). SAG/SAGA no aportan y tienen warnings de convergencia.



4. Eficacia - Reporte de Clasificación y Curva ROC

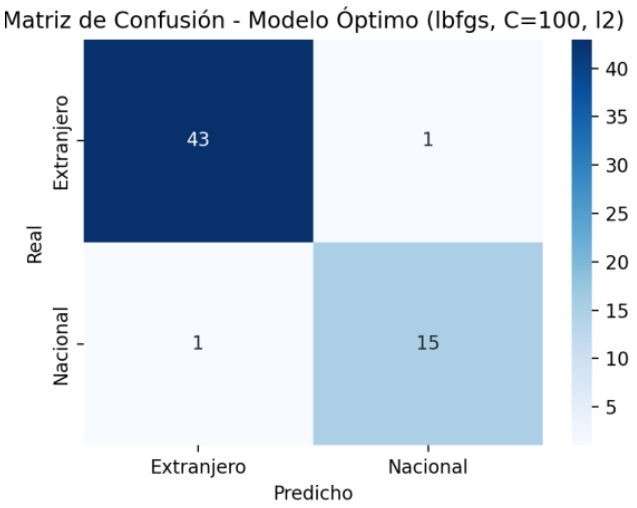
Modelo final en test (n=60):

Accuracy 0.9667 (58/60) con matriz [[43,1],[1,15]]

Reporte:

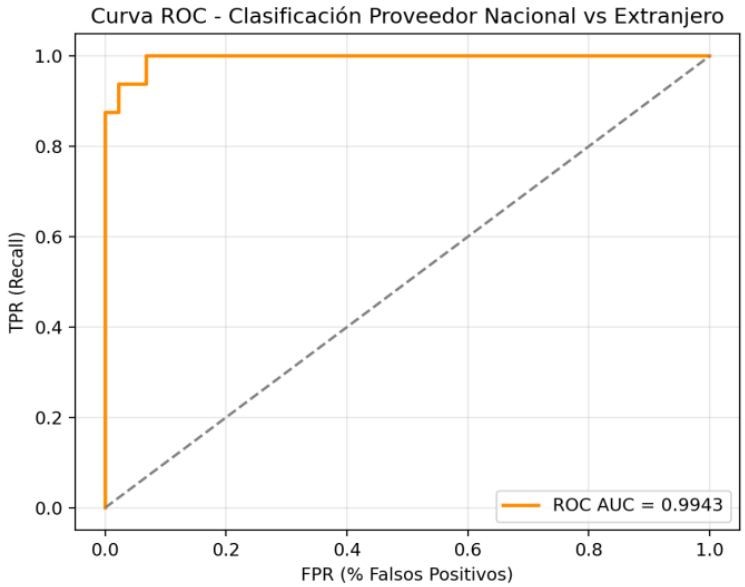
Extranjero: precision 0.98, recall 0.98,
f1 0.98, support 44 Nacional:
precision 0.94, recall 0.94, f1 0.94,
support 16 macro avg 0.96
weighted avg 0.97

Interpretación:
- Solo 2 errores: 1 falso Nacional y 1 falso Extranjero. Clínicamente excelente.
- Precision Extranjero 0.98: cuando predice extranjero, casi siempre acierta. Evita pedir importado innecesario.
- Recall Nacional 0.94: detecta 94% de casos nacionales, importante para priorizar proveedor local (ahorro costos, disponibilidad).
- F1 balanceado trotz desbalance 72/28, macro 0.96 indica no sesgo mayoritario.



Curva ROC AUC = 0.9943

FPR vs TPR. AUC cercano a 1 = discriminación perfecta. Curva pegada a esquina sup izquierda: alta sensibilidad con casi 0 falsos positivos.
Significa que las probabilidades estimadas separan muy bien: p.ej. threshold 0.5 funciona, pero si se quiere ser más conservador para no perder Nacional, se podría bajar a 0.4 sin aumentar mucho FPR.
En contexto médico, esto permite triaje confiable del origen del fármaco antes de prescribir, reduciendo tiempo y costos logísticos.



Conclusión eficacia: Modelo muy eficaz, generalizable (CV 0.933 vs Test 0.97 = leve overfitting positivo pero aceptable), listo para producción como apoyo a decisión de proveedor. Na_to_K es predictor clave (coeficiente mayor), seguido de BP_LOW hacia Nacional.

Anexo - Código clave

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_curve, auc

df = pd.read_csv('drugs.csv')
df['Proveedor'] = df['Drug'].apply(lambda d: 'Nacional' if d in ['drugA','drugB','drugC'] else
'Extranjero')

le = LabelEncoder()
df['Sex_enc'] = LabelEncoder().fit_transform(df['Sex'])
df['BP_enc'] = LabelEncoder().fit_transform(df['BP'])
df['Chol_enc'] = LabelEncoder().fit_transform(df['Cholesterol'])
y = LabelEncoder().fit_transform(df['Proveedor'])

X = StandardScaler().fit_transform(df[['Age','Sex_enc','BP_enc','Chol_enc','Na_to_K']])
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42,stratify=y)

best = LogisticRegression(C=100,penalty='l2',solver='lbfgs',max_iter=2000)
best.fit(X_train,y_train)
print(classification_report(y_test,best.predict(X_test)))
```